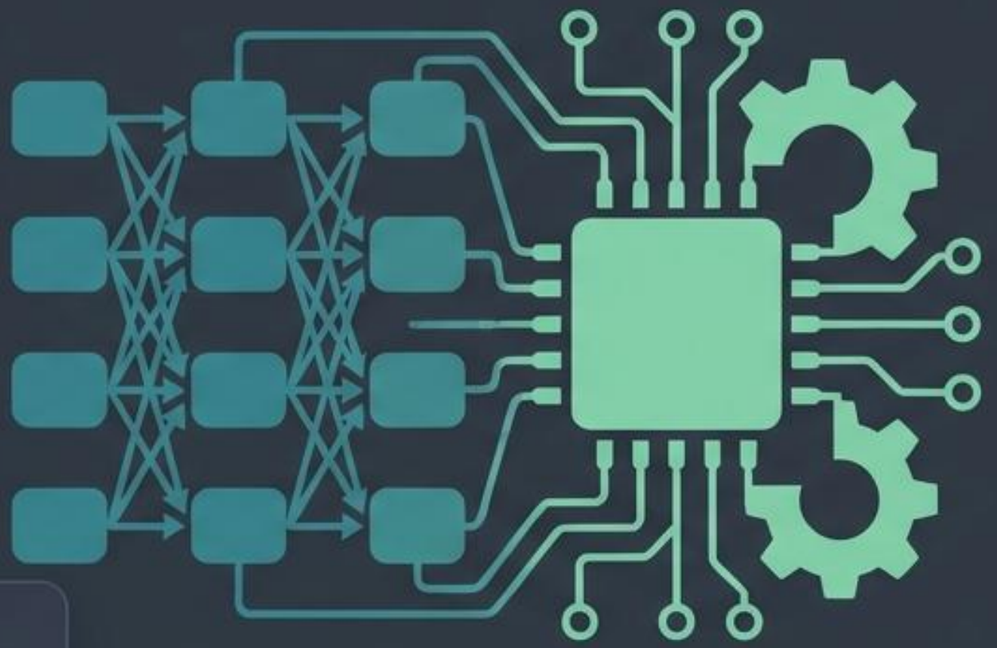
Sesión 08-B: Del Entrenamiento al Despliegue — Evaluación, Versionado y Documentación

Diseño y Construcción de una Solución CNN Completa (Parte 2)



> Curso: Ingeniería del Conocimiento (IS056B)
> Unidad: I – Machine Learning y Redes Neuronales Profundas
> Docente: Msc. Jaime Antonio Huaytalla Pariona
> Institución: Universidad Nacional del Centro del Perú (UNCP), 2026-II

El Puente hacia Producción

Entrenar el modelo es solo el 20% del trabajo. Hoy construiremos el **80% restante: llevarlo al mundo real.**



- Punto de partida: 2 modelos CNN entrenados con callbacks.
- Objetivo de hoy: Pipeline completo en TensorFlow/Keras.
- Enfoque: Rigor de ingeniería de software aplicado a IA.

Fase 6: Suite de Evaluación Completa

código_evaluación.py

```python
y_pred_probs = model.predict(test_dataset)

y_pred = np.argmax(y_pred_probs, axis=1)

print(classification_report(y_true, y_pred))
```

CRÍTICO: Evaluación sobre datos que el modelo NUNCA vio en entrenamiento ni validación.

Output Terminal

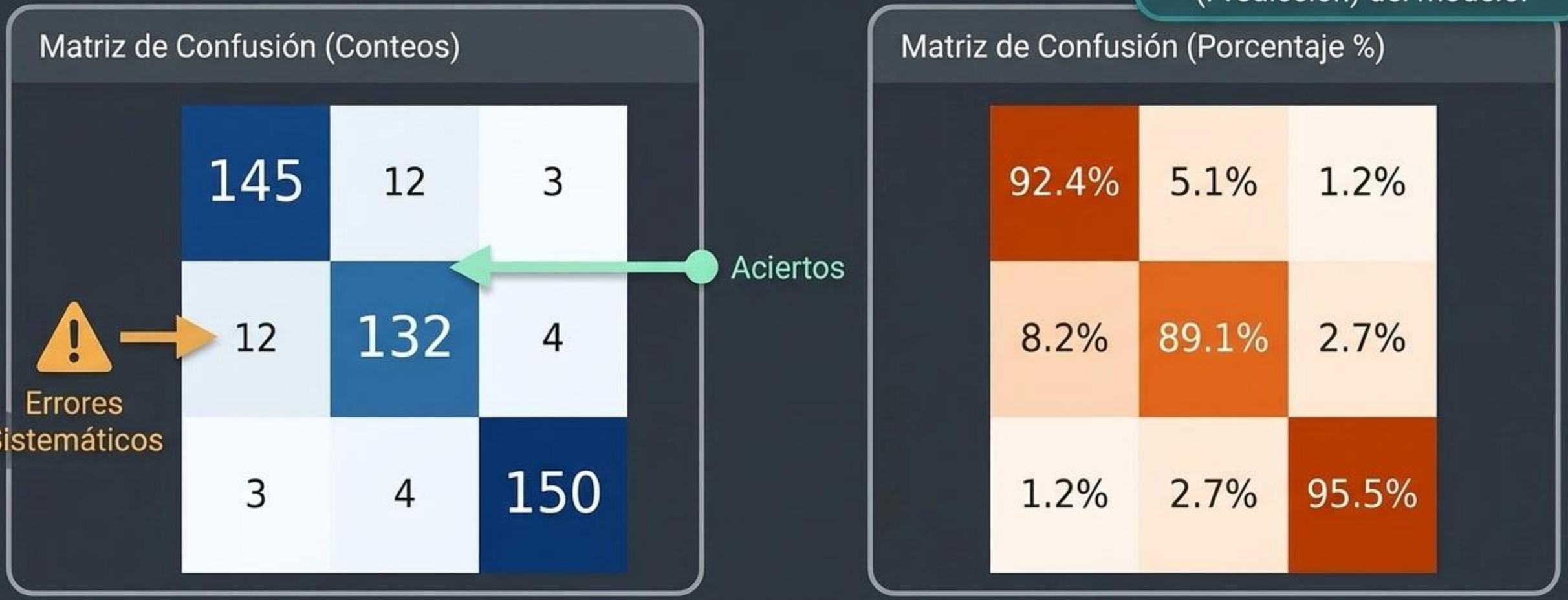
```
              precision    recall  f1-score

     clase_0       0.92      0.89      0.90
     clase_1       0.85      0.87      0.86
     clase_2       0.94      0.96      0.95

    accuracy                           0.91
   macro avg       0.90      0.91      0.90
```

Matriz de Confusión Profesional

¿Cómo leer la matriz?
Filas = La Realidad.
Columnas = La Expectativa (Predicción) del modelo.




```
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')
sns.heatmap(cm_percent, annot=True, fmt='.1f', cmap='Oranges')
```

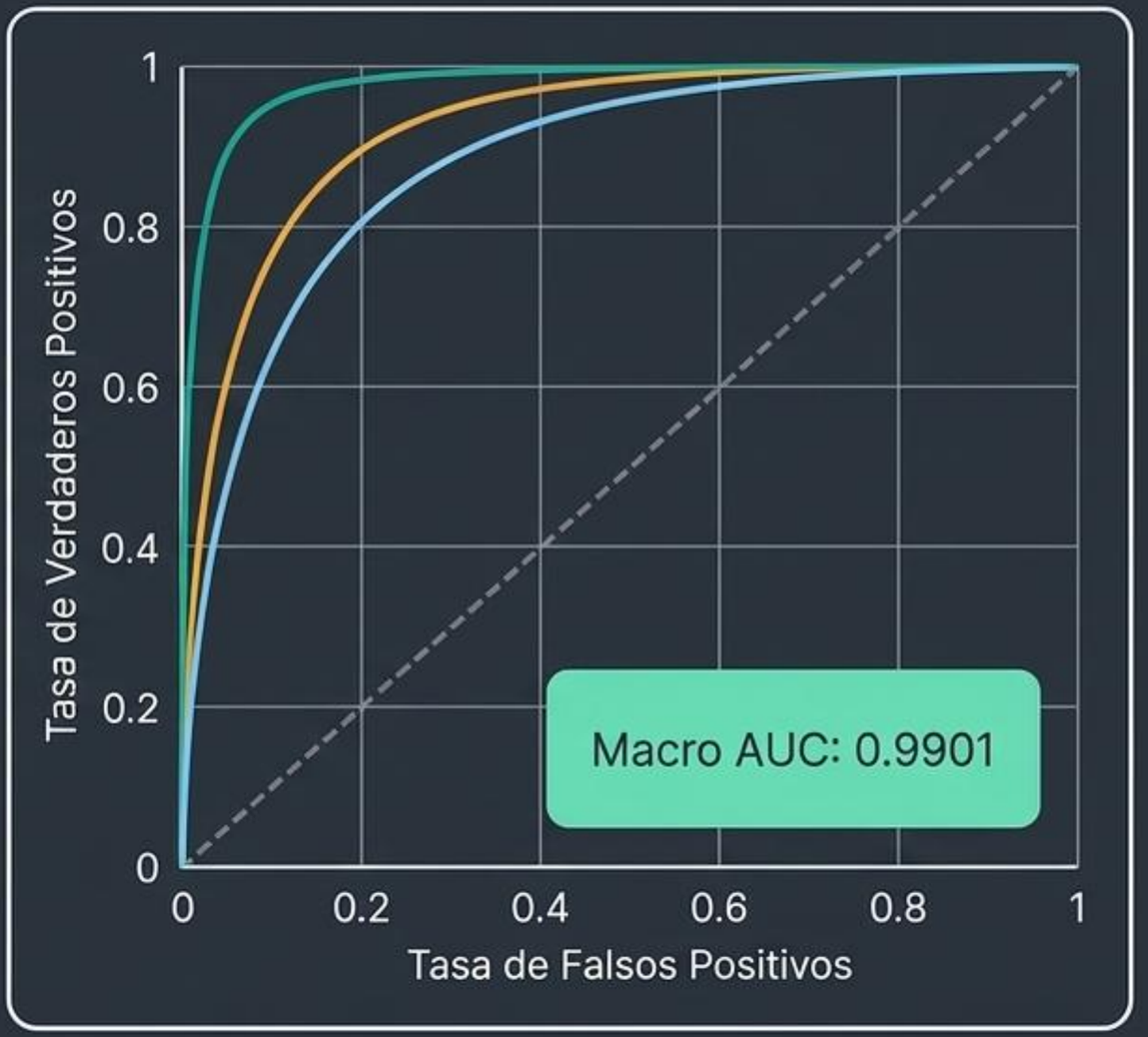
Curvas ROC Multiclase y Macro AUC

Enfoque One-vs-Rest (OvR):

Evalúa la capacidad del modelo para distinguir una clase específica contra todas las demás juntas.

macro_auc_calculo.py

```python
y_true_bin = label_binarize(y_true, classes=[0,1,2])
macro_auc = roc_auc_score(
    y_true_bin,
    y_pred_probs,
    multi_class='ovr',
    average='macro'
)
```



Fase 7: Análisis de Errores — ¿Por Qué Falla?

El análisis de errores transforma un modelo aceptable en uno excelente.

TOP CONFUSIONES DEL MODELO

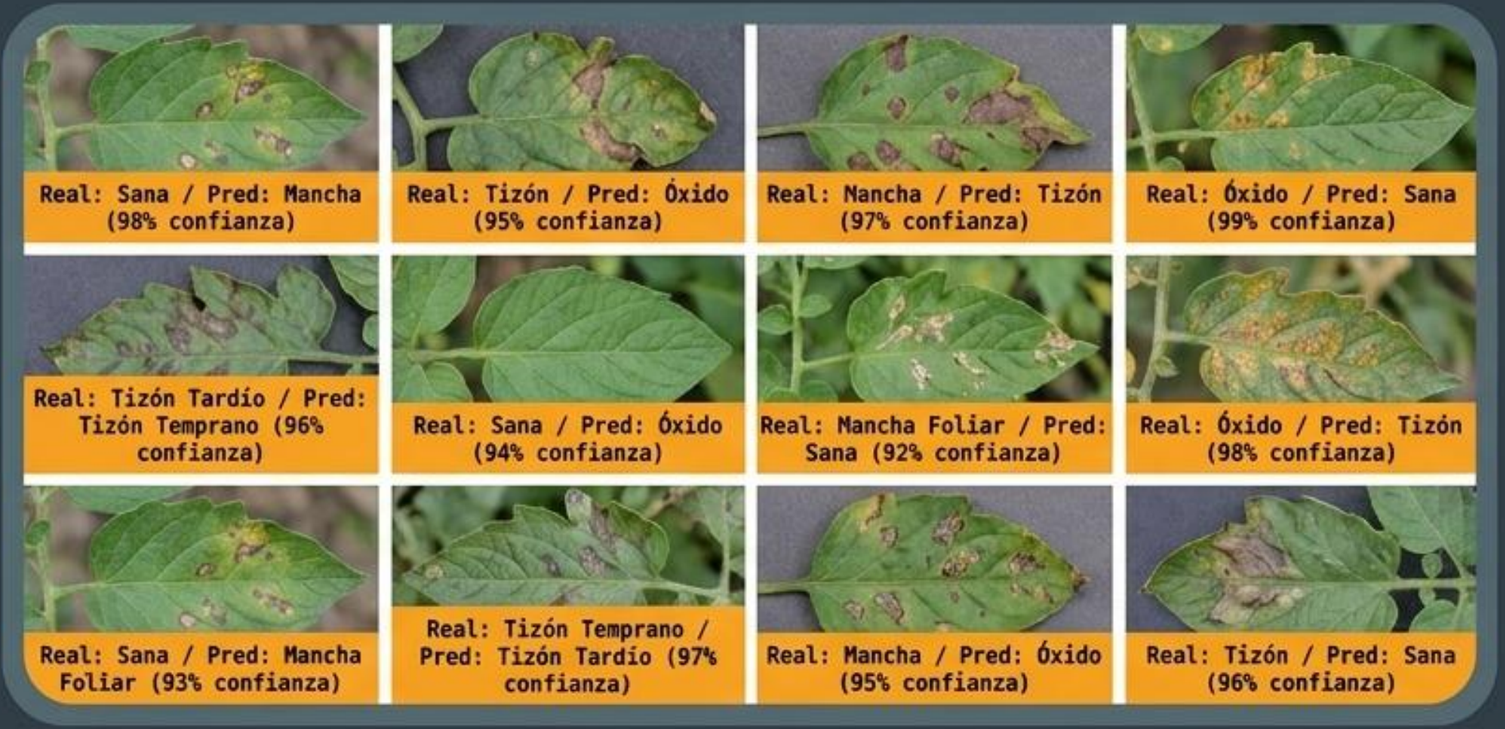
Clase Real	Clase Predicha	Errores (Conteo)	% de la Clase
Tizón Temprano	Tizón Tardío	45	18.5%
Sana	Mancha Foliar	22	9.1%
Óxido	Tizón Temprano	15	6.2%
Mancha Foliar	Sana	8	3.4%

pandas_auc_calculo.py

```
cm = confusion_matrix(y_true, y_pred)
np.fill_diagonal(cm, 0) # Ignorar aciertos
# Extraer pares con mayores valores...
```

Visualización de Errores: Cero Precisión, Alta Confianza

¿Qué revela esto? Identifica limitaciones ocultas (ej. oclusiones, iluminación) y casos donde el modelo está sobre-confiado en su error.



error_extraction_logic.py

```
wrong_idx = np.where(y_true != y_pred)[0]
confidences = y_pred_probs[wrong_idx, y_pred[wrong_idx]]
most_confident_errors = wrong_idx[np.argsort(-confidences)][:12]
```

Estrategias de Mejora Post-Análisis

Diagnostic Matrix

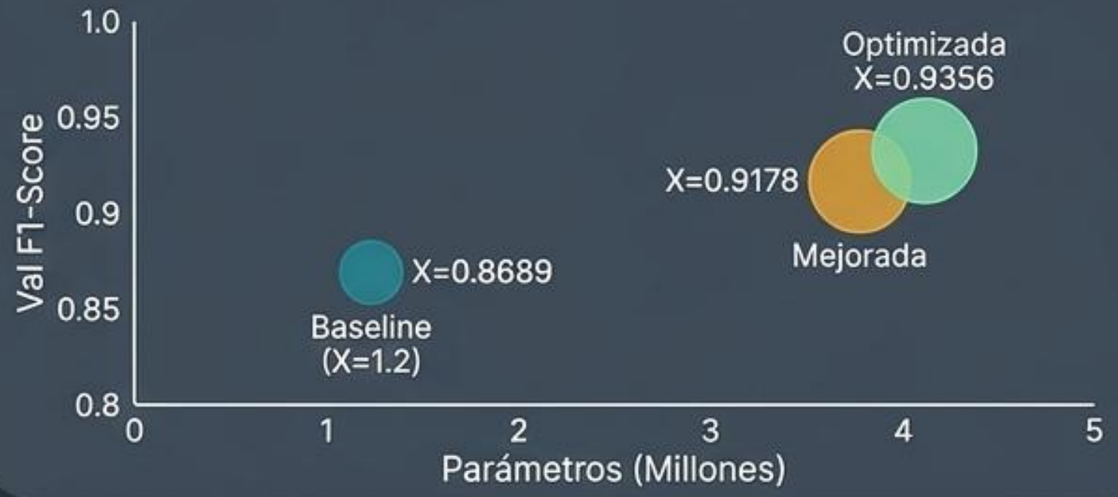
Síntoma (Hallazgo)	Causa Probable	Tratamiento (Acción Correctiva)
Dos clases se confunden mucho	Visualmente similares	Fusionar clases o recolectar más datos específicos.
Errores en imágenes oscuras	Sesgo de iluminación en dataset	Aplicar augmentation de brillo/contraste.
Baja precisión, alto recall	Clase 'atractora' de falsos positivos	Revisar balanceo de datos.
Modelo muy seguro en errores	Posible Overfitting	Calibrar probabilidades o aumentar regularización.

Fase 8: Comparación Sistemática de Modelos

Comparación de Métricas



Trade-off



Takeaway

Trade-off: Un modelo más complejo no siempre es viable si el tiempo de inferencia y el tamaño superan las restricciones de producción.

Fase 9: Versionado del Proyecto (MLOps Básico)

¿Por qué versionar? Reproducibilidad, colaboración en equipo y capacidad de rollback.

```
proyecto_cnn/
├── data/
│   ├→ raw/ (No Git)
│   └── processed/ (No Git)
├── src/
│   ├→ models/
│   └── training/
├── experiments/
└── models/
    └→ production/
        └── model_v1.keras
```

.gitignore

```
1  data/raw/
2  data/processed/
3  *.keras
4  *.h5
```

Regla de Oro: ¡Git es para código, no para pesos! Nunca versionar carpetas de datos crudos ni archivos de modelos pesados. Usar herramientas como DVC para datos.

Metadata del Modelo: El 'DNI' del Experimento

1. Arquitectura y Parámetros

2. Configuración y Métricas

3. Info de Origen

4. Reproducibilidad Técnica

```
{
  "model": {
    "architecture": "CNN_Optimizada",
    "total_params": 4100000
  },
  "training": {
    "best_val_loss": 0.045,
    "final_epoch": 38
  },
  "dataset": {
    "name": "PlantVillage",
    "total_images": 54305
  },
  "environment": {
    "tensorflow_version": "2.15",
    "python_version": "3.9"
  }
}
```

Formato JSON para máxima interoperabilidad entre sistemas.

Fase 10: Logging y Registro de Experimentos



Custom Callback (ExperimentLogger)

experiment.log

```
1  [2023-10-27 14:30:00] INFO: Epoch 1: loss=0.1523, val_acc=0.9451
2  [2023-10-27 14:30:00] INFO: Epoch 2: loss=0.1523, val_acc=0.9451
3  ...
4  [2023-10-27 14:30:00] INFO: Epoch 3: loss=0.1523, val_acc=0.9451
```

TensorBoard




```python
logger.info(
    f'Época {epoch+1}: '
    f'loss={logs['loss']:.4f}, '
    f'val_acc={logs['val_accuracy']:.4f}'
)
```

No depender del output efímero de la consola. Todo experimento debe dejar una huella inmutable.

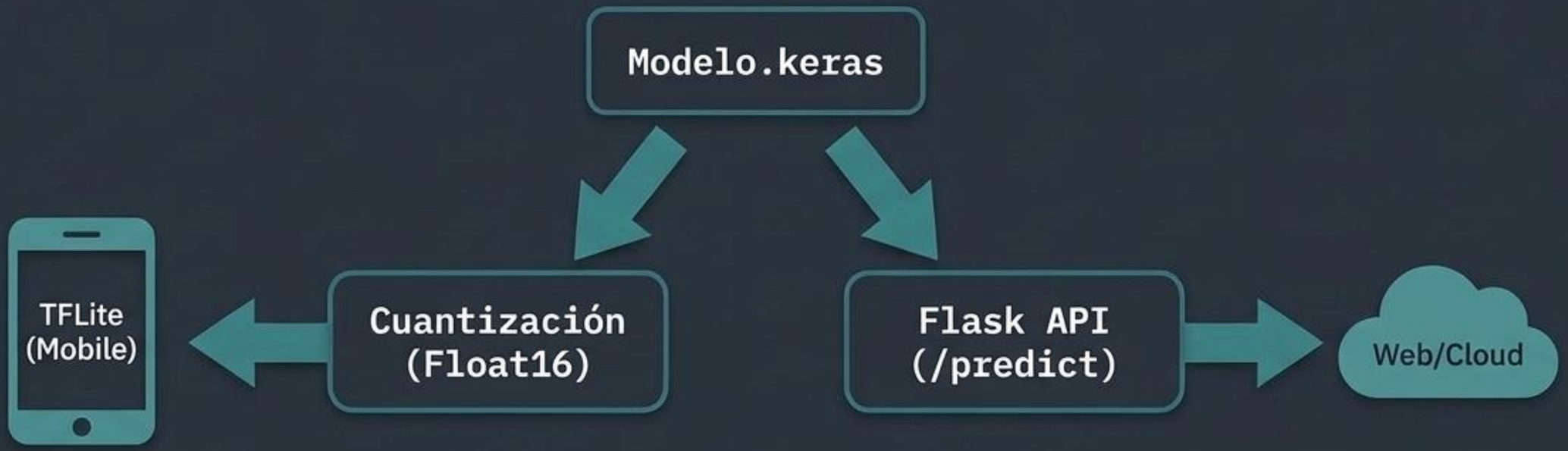
Fase 11: Documentación Técnica

Markdown report (**README.md**)

1. **Resumen Ejecutivo**
2. **Definición del Problema**
3. **Dataset (EDA)**
4. **Metodología y Arquitectura**
5. **Resultados (Métricas)**
6. **Conclusiones**
7. **Reproducibilidad**

El sello de oro de la ingeniería de software. Requiere `requirements.txt`, semillas fijadas (seed) e instrucciones claras de ejecución.

Fase 12: Despliegue Básico



TFLite Export

```
converter.optimizations = [tf.lite.Optimize.DEFAULT]
converter.target_spec.supported_types = [tf.float16]

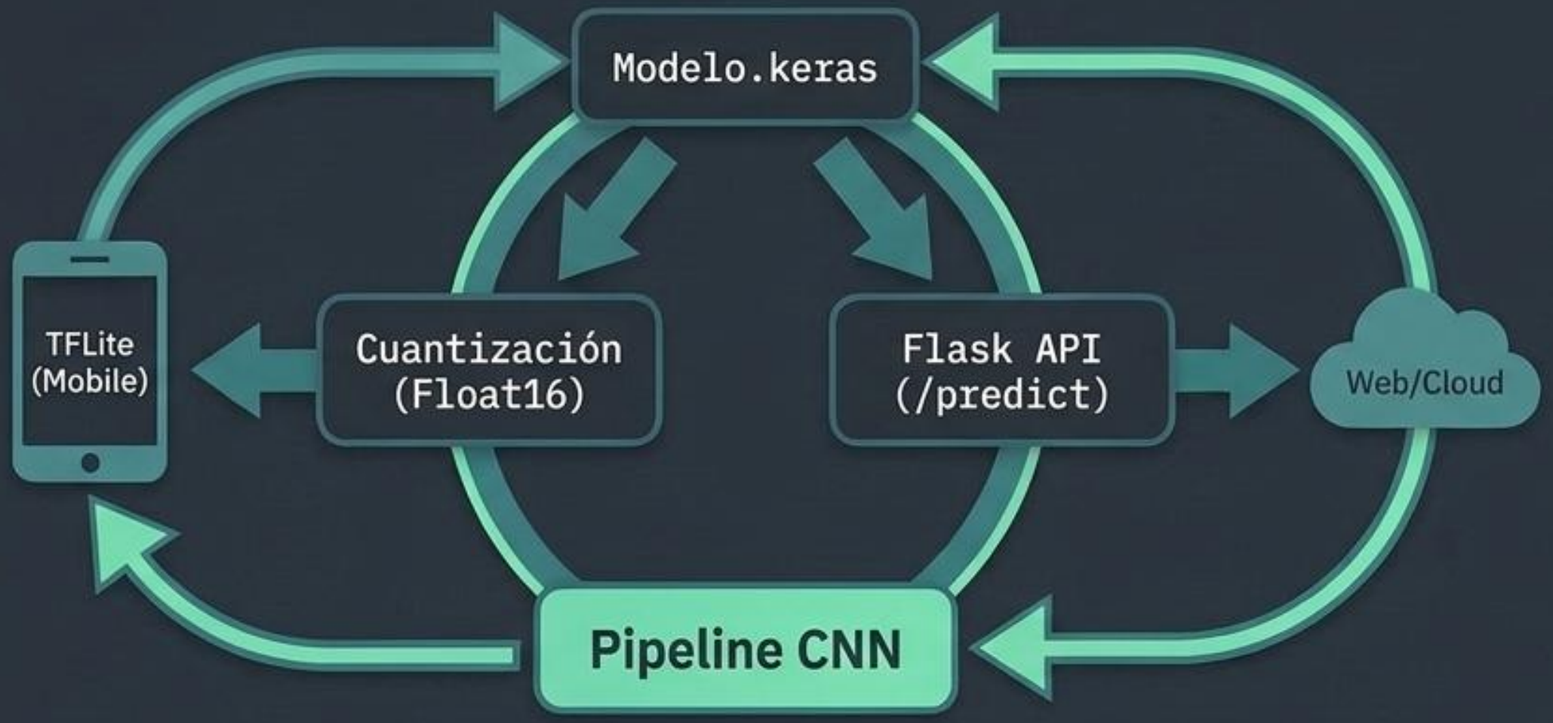
tflite_model = converter.convert()
```

Flask Endpoint

```
@app.route('/predict', methods=['POST'])
def predict():
    predictions = MODEL.predict(img_array)
    return jsonify({'class': predicted_class})
```

El modelo sale del laboratorio. Se vuelve un servicio consumible.

Cierre: Checklist del Pipeline CNN Completo



- ✓ 1. Problema definido
- ✓ 2. Dataset
- ✓ 3. EDA
- ✓ 4. Preprocesamiento
- ✓ 5. Arquitectura (x2)
- ✓ 6. Entrenamiento
- ✓ 7. Evaluación rigurosa
- ✓ 8. Análisis de errores
- ✓ 9. Comparación
- ✓ 10. Versionado/Git
- ✓ 11. Reporte Técnico
- ✓ 12. Script de Despliegue

Próxima Unidad: Transfer Learning, Series de Tiempo y MLOps Avanzado.
Ahora tienen las herramientas para construir soluciones profesionales.